

# Architecture & Trade-offs

In-memory account ledger core | Implementation review | 26 September 2026

## Design and correctness boundary

The core is a single-writer account subledger. Immutable records preserve input decisions and their monetary or authorization effects. Integer minor units and rational interest arithmetic remove ambient decimal-context and floating-point dependence. Currency belongs to the account; mismatches are rejected. The journal is authoritative; balances, holds, and accruals are disposable projections.

Three coordinates matter: arrival sequence, the supplied booked-day label, and the financial value day. A sequence cutoff answers what the system knew at a decision. A value-day cutoff answers when money takes effect. Historical holds are never re-decided using subsequently learned balances. Exact event retries return the first decision; conflicting payloads append an error without moving money. This is process-local idempotency, not a distributed delivery guarantee.

Fee assessments remain after a principal reversal because refund authorization is a separate policy. That choice is independent of append-only storage: a compensating refund would be technically straightforward. Daily interest can change with new knowledge; corrections append deltas instead of replacing prior accruals. A finalization boundary prevents additional monetary changes after capitalization. Production would need a new adjustment period, not silent edits.

## Append-only at scale

The first likely bottleneck at 100x transaction volume is repeated journal scanning. Event deduplication, authorization reconstruction, and balance queries each inspect history. Across  $N$  arrivals, these paths can approach  $O(N^2)$ . A close over  $A$  accounts and  $D$  elapsed days is approximately  $O(A \times D \times N)$ . With  $A$  and  $D$  fixed, a close grows roughly with  $N$ ; multiplying  $N$  by 100 can make the repeated ingest scans approach 10,000 times their previous work. These are complexity estimates, not measured throughput claims.

Memory also grows without bound: original decisions, error attempts, monetary rows, terminal authorization transitions, fees, and accrual revisions all remain. Snapshotting a balance alone does not bound this growth or answer a historical knowledge query. The current tuple journal accessor also copies references; it is convenient for tests, not a bulk-history interface.

The cheapest structural improvement is rebuildable indexes: event ID to first decision, account/auth ID to latest transition, and account/value-day monetary buckets with prefix totals. Invalidate affected suffixes when a backdate arrives. This defers CPU pressure without changing journal semantics, but adds memory and projection-consistency obligations. Later, move sealed journal segments to durable storage and retain hot indexes plus sequence-tagged checkpoints. Replay must still reproduce a checkpoint and subsequent corrections; archiving is not permission to erase customer or audit history.

## Value-dated entries in production

A backdate can change a previously issued statement, fee eligibility, interest, and downstream accounting or reporting. Preserving both the original decision and its later financial correction is essential for explaining a customer complaint. Retaining fees after correcting principal is an exercise policy, not a claim that a UAE bank should charge for its own processing error.

CBUAE Consumer Protection Standards require disclosure of applicable fees and their calculation method and recurrence, and regular detailed account statements (Article 2, clauses 2.1.1.37, 2.1.1.38 and 2.1.1.43) [1]. These requirements make unexplained retrospective charges and inconsistent statements an operational and conduct concern. The current Operational Risk Management Regulation includes approval authorities and segregation of duties among internal controls (Article 7) [2]. These sources support the control objective; they do not mandate this implementation's exact fee-retention or catch-up algorithm.

Before launch I would add a maker-checker backdating gate: preserve an immutable reason and source evidence, show the proposed value date and customer impact, and require a separate authorized approver before a closed-period adjustment can post. A linked reconciliation case would track corrected statements, customer redress and affected downstream reports. Compliance must confirm the product terms, permitted tariff, reporting impact, retention and local data requirements. The toy tariff is not represented as regulatory approval.

## Authorization lifecycle

An ACTIVE hold reserves spending power independently of ledger postings. The model has two non-settlement exits from ACTIVE and one terminal decision before a hold exists. All transitions append records; none removes the original request.

- **CANCELLED:** an explicit CANCEL models a merchant void or operational release. It releases the entire active hold once. Mandate authenticated provenance and retry-safe release; an inactive ID cannot release funds a second time.
- **EXPIRED:** an explicit EXPIRE models a scheme deadline passing without capture. It releases the whole hold. Mandate a reliable scheduler driven by product/scheme deadlines, with replayable, uniquely identified expiry events. That scheduler is absent here; the deliberately failing test exposes a stranded hold when no expiry message arrives.
- **DECLINED:** insufficient available funds ends the request without creating an active hold. Mandate a new request/ID for retry; a later credit does not retroactively approve it. Malformed requests are rejected inputs, not active authorizations and not releases.

There are no other non-settlement exits: neither principal reversal, account balance recovery, nor reaching the reporting boundary cancels a hold. A later presentment for an expired, cancelled, declined or absent authorization posts as an unmatched clearing item with an exception. It does not reopen the old authorization. Production must link that exception to scheme evidence and dispute handling. A single-capture restriction rejects a second presentment under the same account/auth ID; real multi-capture flows need clearing IDs.

## What was cut and why

Each simplification reduces implementation surface while leaving a specific production obligation unresolved.

- **Durability and balanced books.** No storage, recovery log, general-ledger counter-account or external reconciliation. This meets the in-memory scope; a crash loses the journal and external settlement cannot be proved balanced.
- **Concurrent transactions.** Domain validation precedes writes, but related rows have no durable atomic commit and callers are not synchronized. Production needs atomic batches and account-level serialization to prevent double holds, duplicate captures and partially recorded effects.
- **Incremental projections and resource limits.** Scans keep the implementation auditable. No journal retention, amount-size policy, instalment-count cap or bounded error stream is provided. Untrusted traffic could exhaust memory/CPU; ingress validation and rebuildable indexes are required before service use.
- **Business calendar and post-close amendments.** Integer days replace timezones, settlement calendars and cut-offs. Future value dates and monetary input after finalization are rejected. Real late corrections need controlled accounting periods and subsequent interest adjustments, not a reopened frozen result.
- **Fee products and customer redress.** The fixed accounts omit account creation, fee waivers, reversals of fees and automatic refunds. No BHD tariff or FX rate is invented: a negative BHD close fails before derived writes. Product approval must resolve missing tariffs and remediation rather than silently waive them.
- **Full card lifecycle.** No automatic expiry, incremental holds, partial captures, settlement reversals or chargebacks. One capture releases the whole hold. These cuts defer stranded-funds risk, adjustments and dispute workflows; cancellation and expiry events alone are not an operational lifecycle service.
- **Delivery and identity.** Event identity is process-local and payload-exact; logically equivalent decimal strings are still different payloads. There is no durable inbox/outbox or clearing-network reference mapping. Cross-process retries and duplicated network obligations therefore remain unresolved.
- **Security and operations.** No actor identity, permission model, tamper-evident storage, metrics, alerting or exception work queue. Frozen Python records protect normal code paths, not a hostile process. An appended exception is an audit fact, not evidence that someone investigated it.

## Defense and validation

The regression suite verifies historical cutoffs, fee propagation, input-order preservation, correction records, retry behavior, lifecycle transitions and minor-unit conservation. The separate failing test is intentionally executable and is not masked as an expected failure. Tests establish this model's chosen contract; they do not establish bank readiness, regulatory compliance, load capacity or the correctness of another equally plausible settlement contract.

## Primary references

[1] CBUAE, Consumer Protection Standards, Article 2: Disclosure and Transparency.

<https://rulebook.centralbank.ae/en/rulebook/article-2-disclosure-and-transparency>

[2] CBUAE, Operational Risk Management Regulation, C 1/2026, Article (7): Internal Control System. Effective 14 September 2026. Replaces the 2018 regulation and standards; sources checked 26 September 2026.

<https://rulebook.centralbank.ae/en/rulebook/article-7-internal-control-system>